

FGS DEV — Manual EC2 Deployment Runbook (SSM)

Audience: operators deploying FGS **dev** on a **single EC2** host using **AWS Systems Manager** (Session Manager / Run Command).

Region: `us-east-1`

Compose project path: `/opt/fgs`

Compose file: `docker-compose.ec2.yml` (project name `fgs-ec2`)

ECR repository: `fgs/dockers`

Channel: `dev` (image tags `setup-dev`, `user-dev`, ...)

This runbook is derived from repository files only:

Source	Role
<code>deployment/aws/ec2/README.md</code>	EC2 vs local; deploy order; logs
<code>deployment/aws/ec2/bootstrap-ec2.sh</code>	One-time host bootstrap
<code>deployment/aws/ec2/deploy-service.sh</code>	ECR login, pull, recreate one service
<code>deployment/aws/ec2/docker-compose.ec2.yml</code>	Stack definition and dependencies
<code>deployment/aws/ec2/.env.example</code>	Host <code>.env</code> template
<code>deployment/aws/ec2/config/setup-appsettings.example.json</code>	Setup RDS bootstrap config
<code>deployment/aws/ec2/Push-Ec2Files.ps1</code>	Push compose/deploy scripts via SSM
<code>deployment/aws/manual-guide/GITHUB_ACTIONS_CD_EC2.md</code>	IAM, SSM CD, troubleshooting
<code>deployment/aws/manual-guide/EC2_FULL_SETUP_AND_CD.md</code>	Full setup, health checks, checklist
<code>deployment/aws/terraform/iam.tf</code>	EC2 role / SSM session operator policies
<code>.github/workflows/reusable-deploy-ec2.yml</code>	CD SendCommand pattern

Placeholders used below: `ACCOUNT_ID`, `INSTANCE_ID` (`i-...`), `YOUR_RDS_HOST`, passwords, and connection strings — do not commit real secrets.

Do not use `src/Gateway/docker-compose.yml` on EC2. That file is for **local Docker Desktop** only (`fgs-local`).

1. Architecture (dev single host)



Nginx on EC2 listens on **port 80 only** (TLS at ALB). Entrypoint: `/opt/fgs/nginx-http-only-entrypoint.sh`.

2. Prerequisites

Item	Notes
EC2	Amazon Linux 2023 or Ubuntu 22.04+; <code>t3.medium</code> or larger recommended
VPC	Same VPC as RDS (and ALB if used)
Security group	Inbound 80 from ALB (or operator IP); no SSH required if using SSM
Outbound	HTTPS 443 to SSM and ECR endpoints
RDS / Postgres	Reachable from EC2; Setup needs <code>ConnectionStrings:FgsSetup</code>
<code>glo.GloCredential</code>	Global DATABASE, REDIS, RABBITMQ, SENDGRID, ENTRA_EXTERNAL_ID, DATADOG, AWS, etc.
ECR	Private repo <code>fgs/dockers</code> with channel tags already pushed (CI or manual)
Operator workstation	AWS CLI v2 + Session Manager plugin (for CLI sessions / <code>Push-Ec2Files.ps1</code>)

3. IAM — least privilege

3.1 EC2 instance profile (host)

Attach a role trusted by `ec2.amazonaws.com` with:

- 1. Managed policy `AmazonSSMManagedInstanceCore`
- 2. **ECR pull** (inline / Terraform `ecr-pull`):

```

"Version": "2012-10-17"
"Statement": [
  {
    "Sid": "EcrAuth",
    "Effect": "Allow",
    "Action": "ecr:GetAuthorizationToken",
    "Resource": "*"
  },
  {
    "Sid": "EcrPull",
    "Effect": "Allow",
    "Action": [
      "ecr:BatchCheckLayerAvailability",
      "ecr:GetDownloadUrlForLayer",
      "ecr:BatchGetImage",
      "ecr:DescribeRepositories"
    ],
    "Resource": "arn:aws:ecr:us-east-1:ACCOUNT_ID:repository/fgs/docker*"
  }
]

```

3. **KMS decrypt** for Setup credential vault (required when Setup uses the instance profile instead of explicit `AwsCredentials keys`) — Terraform `kms-decrypt-credentials` :

```

"Effect": "Allow",
"Action": [
  "kms:Decrypt",
  "kms:GenerateDataKey",
  "kms:DescribeKey"
],
"Resource": "arn:aws:kms:us-east-1:ACCOUNT_ID:key/YOUR_CMK_ID"

```

4. Optional **S3** when File uses the instance profile (no AccessKey in Global AWS): policy file `deployment/aws/iam-fgs-s3-all-buckets-policy.json` (Terraform `s3-all-buckets`).

Terraform names (when `create_ec2_iam` is enabled): `fgs-<env>-ec2-role` , instance profile `fgs-<env>-ec2-profile` .

3.2 Operator (interactive Session Manager)

Attach policy pattern `fgs-<env>-ssm-session-operator` (Terraform). Actions include:

- `ssm:StartSession` , `ssm:TerminateSession` , `ssm:ResumeSession`
- `ssm:DescribeSessions` , `ssm:GetConnectionStatus` , `ssm:DescribeInstanceInformation`
- `ssm:StartSession` ON `SSM-SessionManagerRunShell` and `arn:aws:ec2:us-east-1:ACCOUNT_ID:instance/*`

Do not put Session Manager permissions on the GitHub CD principal — CD uses **SendCommand** only.

3.3 GitHub Actions / CD principal (optional for manual-only)

Needs ECR **push** (for CI) plus:

```

{
  "Sid": "Ec2DeployViaSsm",
  "Effect": "Allow",
  "Action": "ssm:SendCommand",
  "Resource": [
    "arn:aws:ssm:us-east-1::document/AWS-RunShellScript",
    "arn:aws:ec2:us-east-1:ACCOUNT_ID:instance/*"
  ]
},
{
  "Sid": "SsmCommandResults",
  "Effect": "Allow",
  "Action": [
    "ssm:GetCommandInvocation",
    "ssm:ListCommands",
    "ssm:ListCommandInvocations"
  ],
  "Resource": "*"
}

```

4. Connect with SSM

4.1 AWS Console

1. **EC2** → select instance → **Connect** → **Session Manager** → **Connect**.
2. Or **Systems Manager** → **Session Manager** → **Start session**.

Confirm instance is **Online** under Systems Manager → Fleet Manager / Managed instances.

4.2 AWS CLI (operator laptop)

```
aws ssm start-session --target INSTANCE_ID --region us-east-1
```

Requires Session Manager plugin and the operator IAM policy above.

4.3 Non-interactive Run Command (same pattern as CD)

```

aws ssm send-command \
  --instance-ids INSTANCE_ID \
  --document-name AWS-RunShellScript \
  --region us-east-1 \
  --parameters 'commands=["sudo /opt/fgs/deploy-service.sh user-service dev fgs/docker us-east-1"]' \
  --query Command.CommandId --output text

```

Poll:

```

aws ssm get-command-invocation \
  --command-id COMMAND_ID \
  --instance-id INSTANCE_ID \
  --region us-east-1

```

5. Bootstrap /opt/fgs (one time per host)

Bootstrap installs Docker (Amazon Linux / Ubuntu paths in script), creates /opt/fgs , and installs compose/deploy files. **CD does not bootstrap.**

5.1 Option A — Git clone on the instance

```
# Amazon Linux
sudo yum install -y git

# Ubuntu: sudo apt install -y git

cd /tmp
git clone https://github.com/YOUR_ORG/YOUR_REPO.git fgs
cd fgs
sudo bash deployment/aws/ec2/bootstrap-ec2.sh
```

5.2 Option B — Copy deployment/aws/ec2 files, then bootstrap

Required files from deployment/aws/ec2/ :

File	Installed to
bootstrap-ec2.sh	run in place
deploy-service.sh	/opt/fgs/deploy-service.sh (0755)
docker-compose.ec2.yml	/opt/fgs/docker-compose.ec2.yml (0644)
nginx-http-only-entrypoint.sh	/opt/fgs/nginx-http-only-entrypoint.sh (0755)

From a Windows workstation (SSM SendCommand, no SSH):

```
.\deployment\aws\ec2\Push-Ec2Files.ps1 -InstanceId INSTANCE_ID -Region us-east-1
```

Push-Ec2Files.ps1 uploads docker-compose.ec2.yml , deploy-service.sh , and nginx-http-only-entrypoint.sh to /opt/fgs . Still run bootstrap-ec2.sh once for Docker install and placeholder config (or install Docker manually and create config as below).

5.3 Files after bootstrap

Path	Purpose
/opt/fgs/deploy-service.sh	ECR login + pull + compose up -d --no-deps
/opt/fgs/docker-compose.ec2.yml	Stack
/opt/fgs/nginx-http-only-entrypoint.sh	Nginx :80
/opt/fgs/config/setup-appsettings.json	FgsSetup connection string only
/opt/fgs/.env	Host env (RabbitMQ boot, channel, public URL, ...)

Do **not** create /opt/fgs/config/user-appsettings.json . User and other APIs load secrets via Setup credential distribution.

6. Configuration

6.1 Setup RDS bootstrap

```
sudo nano /opt/fgs/config/setup-appsettings.json
```

Template (config/setup-appsettings.example.json):

```
{
  "ConnectionStrings": {
    "FgsSetup": "Host=YOUR_RDS_HOST;Port=5432;Database=fgs_dev_db;Username=YOUR_USER;Password=YOUR_PASSWORD"
  }
}
```

6.2 Host .env

```
sudo nano /opt/fgs/.env
```

From .env.example (replace placeholders):

```
FGS_CONFIG_DIR=/opt/fgs/config
FGS_ECR_REGISTRY=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com
FGS_ECR_REPO=fgs/docker
FGS_CHANNEL=dev

SPNETCORE_ENVIRONMENT=Development

RABBITMQ_USER=fgs
RABBITMQ_PASSWORD=CHANGE_ME_STRONG_PASSWORD

CREDENTIAL_DISTRIBUTION_KEY=fgs-internal-credential-distribution-key

FGS_PUBLIC_BASE_URL=http://YOUR_PUBLIC_HOST_OR_ALIAS
FGS_PUBLIC_SERVICE_PATH=user-service

ID_ENV=dev
ID_SITE=datadoghq.com
```

deploy-service.sh upserts FGS_*_IMAGE lines to channel tags when you deploy.

6.3 RabbitMQ two-layer model

Location	Purpose
/opt/fgs/.env RABBITMQ_USER / RABBITMQ_PASSWORD	RabbitMQ container boot (RABBITMQ_DEFAULT_*)
glo.GloCredential Global:RABBITMQ	Setup reads Username/Password (or ConnectionUri) at startup

Passwords must match. Compose sets RabbitMq__HostName=rabbitmq for Setup; it does **not** inject RabbitMq__Password from .env (that would override the credential table).

6.4 Credentials in RDS (glo.GloCredential)

Ensure Global providers exist for consumers, including at least:

- DATABASE — FgsUser, FgsAudit, FgsNotification, FgsSetup, publisher outbox DBs as needed
- REDIS — use host redis:6379 on this compose network
- RABBITMQ — match .env
- SENDGRID, ENTRA_EXTERNAL_ID, DATADOG, AWS, etc.

7. ECR authentication and images

deploy-service.sh performs ECR login on every deploy:

```
aws ecr get-login-password --region us-east-1  
| docker login --username AWS --password-stdin "$REGISTRY"
```

Registry is resolved via `aws ecr describe-repositories --repository-names fgs/dockers`.

Channel tags (dev):

setup-dev, audit-dev, user-dev, bff-dev, notification-dev, file-dev, publisher-dev, consumer-dev, nginx-dev, redis-dev, rabbitmq-dev

CI also pushes immutable tags such as `setup-<version>-dev-<sha>` (see OIDC/ECR guides). Day-to-day EC2 deploy uses **channel** tags via `deploy-service.sh`.

Images must exist in ECR **before** first pull (merge to `dev` with version bumps, or workflow dispatch with push enabled).

8. Deployment order

8.1 Full first stack (confirmed bootstrap / README)

```
cd /opt/fgs  
sudo ./deploy-service.sh redis dev  
sudo ./deploy-service.sh rabbitmq dev  
sudo ./deploy-service.sh setup-service dev  
sudo ./deploy-service.sh audit-service dev  
sudo ./deploy-service.sh user-service dev  
sudo ./deploy-service.sh bff-service dev  
sudo ./deploy-service.sh notification-service dev  
sudo ./deploy-service.sh file-service dev  
sudo ./deploy-service.sh publisher-service dev  
sudo ./deploy-service.sh consumer-service dev  
sudo ./deploy-service.sh nginx dev
```

Explicit region/repo (same as CD):

```
sudo ./deploy-service.sh setup-service dev fgs/dockers us-east-1
```

What each call does (`deploy-service.sh`):

1. ECR login
2. Upsert `.env` image variables for the channel
3. `docker compose -f docker-compose.ec2.yml pull <service>`

4. `docker compose -f docker-compose.ec2.yml up -d --no-deps <service>`
5. `docker compose ... ps <service>`

8.2 Core path: Redis → RabbitMQ → Setup → User → NGINX

Compose dependencies (important):

Service	depends_on (healthy)
setup-service	redis , rabbitmq
user-service	redis , setup-service
nginx	setup-service , user-service , bff-service

So a **working nginx container** as defined in `docker-compose.ec2.yml` also expects **bff-service** healthy.
Recommended core sequence:

```

sudo ./deploy-service.sh redis dev
sudo ./deploy-service.sh rabbitmq dev
sudo ./deploy-service.sh setup-service dev
sudo ./deploy-service.sh user-service dev
sudo ./deploy-service.sh bff-service dev
sudo ./deploy-service.sh nginx dev

```

(`deploy-service.sh` uses `--no-deps` , so you can force-start nginx alone, but health/routing for BFF will fail until `bff-service` is up.)

Optional infrastructure-only prelude:

```

cd /opt/fgs
docker compose -f docker-compose.ec2.yml up -d redis rabbitmq
# wait until healthy, then deploy-service for apps

```

8.3 Normal (day-2) deploy vs fresh vs rollback

Scenario	Actions
Normal (one service after CI)	<code>sudo /opt/fgs/deploy-service.sh <compose-service> dev fgs/dockers us-east-1</code> — same as GitHub CD
Fresh host (new/resized EC2)	New instance profile → SSM Online → bootstrap → restore <code>setup-appsettings.json</code> + <code>.env</code> → update GitHub <code>EC2_INSTANCE_ID</code> → full order §8.1
Rollback	Channel tags are mutable. Prefer pinning an immutable ECR tag in <code>/opt/fgs/.env</code> (e.g. <code>FGS_USER_IMAGE=.../fgs/dockers:user-<version>-dev-<sha></code>), then <code>docker compose -f docker-compose.ec2.yml up -d --no-deps user-service</code> . Or redeploy a known-good build that retags <code>*-dev</code> .

9. Restart, logs, health checks

9.1 Status

```
cd /opt/fgs
docker compose -f docker-compose.ec2.yml ps
```

9.2 Restart / recreate one service

Prefer deploy script (pulls latest channel tag):

```
sudo /opt/fgs/deploy-service.sh file-service dev
```

Force recreate without changing script:

```
cd /opt/fgs
docker compose -f docker-compose.ec2.yml up -d --force-recreate --no-deps file-service
```

9.3 Logs

```
docker logs fgs-ec2-setup-service-1 --tail 100
docker logs fgs-ec2-audit-service-1 --tail 100
docker logs fgs-ec2-user-service-1 --tail 100
docker logs fgs-ec2-notification-service-1 --tail 100
docker logs fgs-ec2-file-service-1 --tail 100
docker logs fgs-ec2-publisher-service-1 --tail 100
docker logs fgs-ec2-consumer-service-1 --tail 100

or by name filter
docker logs $(docker ps -qf name=setup-service) --tail 100
docker logs $(docker ps -qf name=nginx) --tail 50
```

9.4 Health checks (on instance)

```
docker exec $(docker ps -qf name=redis) redis-cli ping
Expected: PONG

docker exec $(docker ps -qf name=rabbitmq) rabbitmq-diagnostics -q ping

curl -s -o /dev/null -w "%{http_code}\n" http://localhost/nginx-health
Expected: 200

docker exec $(docker ps -qf name=setup-service) curl -fsS http://localhost:5004/health
docker exec $(docker ps -qf name=user-service) curl -fsS http://localhost:5001/health

curl -s -o /dev/null -w "%{http_code}\n" http://localhost/swagger/setup/
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/swagger/user/
```

ALB target group health path: `/nginx-health`, matcher **200**, instance port **80**.

9.5 Verify images in use

```
docker inspect $(docker ps -qf name=setup-service) --format '{{.Config.Image}}'  
docker inspect $(docker ps -qf name=user-service) --format '{{.Config.Image}}'  
docker inspect $(docker ps -qf name=nginx) --format '{{.Config.Image}}'
```

Expected pattern:

```
ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/fgs/dockers:setup-dev
```

10. Docker image cleanup (host)

deploy-service.sh **pulls** new images; previous digests can remain on disk. The repo does **not** ship an automated prune script on EC2.

Operator steps after a successful deploy:

```
docker images  
docker image prune -f  
optional deeper cleanup (removes unused images — confirm nothing needed first)  
docker image prune -a -f
```

ECR-side retention (shared repo lifecycle) is documented separately in the manual ECR/OIDC guides — not part of the EC2 host script.

11. Fresh EC2 after vertical scaling / recreation

When the instance is **replaced** (new instance ID, same or larger type):

1. Launch EC2 with the **same IAM instance profile** (SSM + ECR pull + KMS/S3 as required).
2. Security group: inbound 80 from ALB; outbound 443.
3. Wait until SSM shows **Online**.
4. Session Manager → run `bootstrap-ec2.sh` (git clone or copy files).
5. Restore `/opt/fgs/config/setup-appsettings.json` and `/opt/fgs/.env` (from secure backup — not git).
6. Optionally refresh scripts: `Push-Ec2Files.ps1 -InstanceId NEW_INSTANCE_ID`.
7. Update GitHub repository variable `EC2_INSTANCE_ID` to the new id.
8. Confirm ECR channel tags exist; run **full deploy order** (§8.1).
9. Retarget ALB target group to the new instance if applicable.
10. Verify `/nginx-health` and service health (§9.4).

Data volumes (`redis-data` , `rabbitmq-data`) are **local to the instance** — a new host starts empty brokers unless you restore volumes separately (not automated in repo scripts).

12. Troubleshooting

Symptom	Likely cause	Fix
Cannot StartSession	Instance not registered / no SSM role / no outbound 443	Fix instance profile and SG; confirm Online
SSM SendCommand AccessDenied	CD/operator missing SendCommand policy	Add §3.3 policy
SSM Failed: missing <code>deploy-service.sh</code>	Bootstrap not run	Run <code>bootstrap-ec2.sh</code>
ECR pull denied	Instance role missing ECR pull	Attach §3.1 ECR policy
Setup crash / unhealthy	Bad <code>setup-appsettings.json</code> RDS string	Edit FgsSetup connection string
Setup "Loaded 0 credential" / 503 on resolved	Missing <code>kms:Decrypt</code> on CMK for instance role	Attach KMS policy §3.1
RabbitMQ auth errors	<code>.env</code> password $\neq$ GloCredential RABBITMQ	Align passwords
nginx 502	Upstreams not healthy	Fix setup/user/bff first; <code>docker compose ps</code>
nginx won't stay healthy on empty stack	<code>depends_on</code> includes bff	Deploy <code>bff-service</code> before relying on nginx health
CD skipped	No version bump or PR-only build	Merge to <code>dev</code> with version change
Missing <code>EC2_INSTANCE_ID</code>	GitHub variable not set	Settings → Actions → Variables

View Run Command output: **Systems Manager** → **Run Command** → select command → stdout/stderr.

13. Rollback procedure (manual)

1. Identify a known-good **immutable** tag in ECR (e.g. `user-1.0.12-dev-abc1234`).
2. On EC2:

```
sudo nano /opt/fgs/.env
set e.g. FGS_USER_IMAGE=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/fgs/docker:user-1.0.12-dev-abc1234

cd /opt/fgs
aws ecr get-login-password --region us-east-1 \
| docker login --username AWS --password-stdin ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com
docker compose -f docker-compose.ec2.yml pull user-service
docker compose -f docker-compose.ec2.yml up -d --no-deps user-service
docker compose -f docker-compose.ec2.yml ps user-service
```

3. Note: the next `deploy-service.sh user-service dev` run will **overwrite** `.env` back to the channel tag `user-dev`.

14. Sync compose files without full bootstrap

```
\deployment\aws\ec2\Push-Ec2Files.ps1 -InstanceId INSTANCE_ID -Region us-east-1
```

Then recreate affected services (especially `nginx` after entrypoint/compose changes):

```
sudo /opt/fgs/deploy-service.sh nginx dev
```

15. Quick-reference checklist

AWS

- [] ECR repo `fgs/dockers` with `*-dev` tags present
- [] EC2 role: `AmazonSSMManagedInstanceCore` + ECR pull (+ KMS / S3 as needed)
- [] Instance profile attached; SSM **Online**
- [] SG: inbound 80 (ALB/IP); outbound 443
- [] RDS reachable from EC2
- [] Optional ALB → instance :80, health `/nginx-health`

Host bootstrap

- [] `bootstrap-ec2.sh` completed
- [] `/opt/fgs/docker-compose.ec2.yml`, `deploy-service.sh`, `nginx` entrypoint present
- [] `setup-appsettings.json` — FgsSetup only
- [] `.env` — RabbitMQ password matches `GloCredential`; `FGS_PUBLIC_BASE_URL` set
- [] `GloCredential` providers populated in RDS

First deploy order

- [] `redis` → `rabbitmq` → `setup` → `audit` → `user` → `bff` → `notification` → `file` → `publisher` → `consumer` → `nginx`

Verify

- [] `docker compose -f /opt/fgs/docker-compose.ec2.yml ps` — healthy
- [] `http://localhost/nginx-health` → 200
- [] Setup / User `/health` OK
- [] Image inspect shows `.../fgs/dockers:*-dev`

Operator / CD

- [] Session Manager works for operators
 - [] GitHub `EC2_INSTANCE_ID` / `AWS_REGION` / `ECR_REPO` set if using CD
 - [] After instance replace: new `EC2_INSTANCE_ID` + full redeploy
-

16. Command cheat sheet

```
# Session
aws ssm start-session --target INSTANCE_ID --region us-east-1

# Deploy one service (normal)
sudo /opt/fgs/deploy-service.sh <service> dev fgs/dockers us-east-1

# Status / logs / health
cd /opt/fgs && docker compose -f docker-compose.ec2.yml ps
docker logs $(docker ps -qf name=setup-service) --tail 100
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/nginx-health

# Push files from laptop
.\deployment\aws\ec2\Push-Ec2Files.ps1 -InstanceId INSTANCE_ID
```

Compose services accepted by `deploy-service.sh` :

`redis`, `rabbitmq`, `setup-service`, `audit-service`, `user-service`, `bff-service`, `notification-service`, `file-service`,
`publisher-service`, `consumer-service`, `nginx`

Document generated from FGS repository deployment guides and scripts. Prefer the source files listed in the header if behavior and this runbook ever diverge.